

Melhores Frameworks para Agents

quais os melhores frameworks pra criação de agents do momento ?

RouteLLM

A escolha do framework ideal para criação de agentes de IA em 2026 depende muito do seu objetivo: se você busca um projeto pronto para produção (enterprise), uma equipe de agentes colaborativos ou algo minimalista e focado em código.

Atualmente, os frameworks mais relevantes são:

1. LangGraph (Ecossistema LangChain)

É amplamente considerado o "padrão ouro" para aplicações empresariais e complexas. Diferente do LangChain tradicional que era linear, o LangGraph permite criar fluxos cíclicos (grafos), o que é essencial para agentes que precisam tentar novamente, corrigir erros ou manter um estado persistente.

- **Destaque:** Controle total sobre o fluxo, suporte nativo a "Human-in-the-loop" (pausar para aprovação humana) e excelente observabilidade via LangSmith.
- **Ideal para:** Sistemas robustos, workflows industriais e agentes que precisam de memória de longo prazo. [DataCamp](#)

2. CrewAI

Focado na colaboração entre múltiplos agentes baseados em "papéis" (Roles). É muito fácil de configurar: você define um "Pesquisador", um "Redator" e um "Gerente", dá metas a eles e o framework orquestra a execução.

- **Destaque:** Simplicidade e rapidez para criar sistemas multi-agentes. Possui uma sintaxe muito amigável e foca na entrega de resultados através de tarefas bem definidas.
- **Ideal para:** Automação de marketing, pesquisa de mercado, escrita de blogs e prototipagem rápida. [No-Code Startup](#)

3. Microsoft AutoGen

Este framework foca na "conversa" entre agentes. Eles resolvem tarefas debatendo entre si e podem até gerar e executar código de forma autônoma para resolver problemas técnicos.

- **Destaque:** Excelente para tarefas de programação e análise de dados. Permite configurações complexas de conversação (agentes que revisam o código de outros, por exemplo).
- **Ideal para:** Engenharia de software assistida por IA e análise de dados complexa. [Microsoft](#)

4. SmolAgents (Hugging Face)

Lançado recentemente pela Hugging Face, este framework é extremamente leve e foca na execução de código Python em vez de trocas complexas de JSON. Ele assume que o LLM pode escrever o código necessário para usar ferramentas.

- **Destaque:** Minimalista (menos de 1.000 linhas de código no núcleo), seguro (sandbox para execução de código) e focado em modelos open-source.
- **Ideal para:** Desenvolvedores que preferem interações diretas via código e eficiência. [Hugging Face](#)

5. PydanticAI

Uma estrela em ascensão focada em **Type-Safety** (segurança de tipos). Ele utiliza a popular biblioteca Pydantic para garantir que as entradas e saídas dos agentes sejam validadas rigorosamente, reduzindo erros de formato e "alucinações" estruturais.

- **Destaque:** Integração perfeita com o ecossistema moderno de Python e validação de dados em tempo real.
- **Ideal para:** Aplicações críticas onde a estrutura dos dados retornados pelo agente não pode falhar. [PydanticAI](#)

Resumo de escolha:

- **Quer controle e produção?** Vá de **LangGraph**.
- **Quer montar uma equipe rápida de "funcionários digitais"?** Vá de **CrewAI**.
- **Seu foco é resolver bugs ou dados via chat?** Vá de **AutoGen**.

- **Prefere algo leve, moderno e tipado?** Tente **PydanticAI** ou **SmolAgents**.

Para mais detalhes sobre tendências de produção, você pode consultar análises de mercado como as do [Alice Labs](#) ou [Towards AI](#).

tenho um problema de classificação de fatura, porem tem um problema que nao existe modelo que va classificar certos estabelecimentos tanto por conta da variedade muito grande de estabelecimentos quanto a dificuldade em classificacao por LLMs

RouteLLM

Esse é um problema muito comum em sistemas financeiros. Na prática, tentar resolver isso apenas com um LLM geralmente não escala bem.

O problema normalmente é este:

- Existem milhões de estabelecimentos.
- O nome na fatura costuma ser ruim ("PAG*ABC123", "MERCADO XPTO LTDA", "PIX JOAO").
- O mesmo estabelecimento pode aparecer com dezenas de nomes diferentes.
- Muitos estabelecimentos vendem produtos de categorias completamente diferentes (Amazon, Mercado Livre, Magalu, postos de gasolina com conveniência etc.).
- Mesmo um LLM muito bom simplesmente não possui conhecimento suficiente para classificar um estabelecimento desconhecido.

Por isso, sistemas de bancos e fintechs normalmente usam uma arquitetura em camadas.

1. Normalização

Antes de qualquer IA, limpe o nome.

Exemplo:

"IFOOD *BURGER KING RJ"

↓

"burger king"

"AMZN MKTPLACE"

↓

"amazon"

"PG *SUPERMERCADO SAO JOAO LTDA"

↓

"supermercado sao joao"

Só essa etapa aumenta bastante a acurácia.

1. Base de conhecimento

A melhor fonte de verdade costuma ser uma base própria.

Por exemplo:

burger king

→ Alimentação

amazon

→ Marketplace

shell

→ Combustível

netflix

→ Streaming

Com alguns meses de uso você terá milhares de estabelecimentos classificados.

Essa base costuma resolver 70–95% das transações.

1. Busca semântica

Se o estabelecimento não existir exatamente, procure por similares.

Exemplo usando embeddings:

BURGER KING COPACABANA

↓

embedding

↓

Top 5 semelhantes

Burger King

Burger King RJ

BK Shopping

...

Se a similaridade for alta (>0.95 , por exemplo), reutilize a classificação.

Isso reduz bastante o trabalho do LLM.

1. Enriquecimento via Web

Quando for realmente desconhecido:

POSTO CAMPEAO LTDA

um agente pode:

- pesquisar no Google
- consultar Google Maps
- OpenStreetMap
- Receita Federal
- CNPJ
- Yelp
- Apple Maps

e descobrir

Posto Campeão

Tipo:

Posto de combustível

Aí grava na base.

Esse estabelecimento nunca mais precisará ser pesquisado.

1. LLM apenas quando necessário

Em vez de perguntar

Classifique "XYZ COMERCIAL LTDA"

pergunte

Nome:

XYZ COMERCIAL LTDA

Descrição encontrada:

Empresa especializada em venda de equipamentos médicos.

Site:

...

Google Maps:

...

Categoria possível:

...

Aí o LLM apenas escolhe a categoria.

Isso funciona muito melhor.

1. Aprendizado contínuo

Depois da classificação:

XYZ COMERCIAL LTDA

↓

Equipamentos médicos

↓

salva

Na próxima transação:

XYZ COMERCIAL LTDA

↓

cache

↓

fim

O sistema melhora continuamente.

1. Quando nem a internet resolve

Existem casos impossíveis.

Exemplo:

PIX MARIA

PIX JOAO

PAG*12345

TRANSFERENCIA

MERCADO 01

Não existe informação suficiente.

Nenhum modelo consegue inferir corretamente sem contexto adicional.

Nesses casos, o correto é retornar algo como:

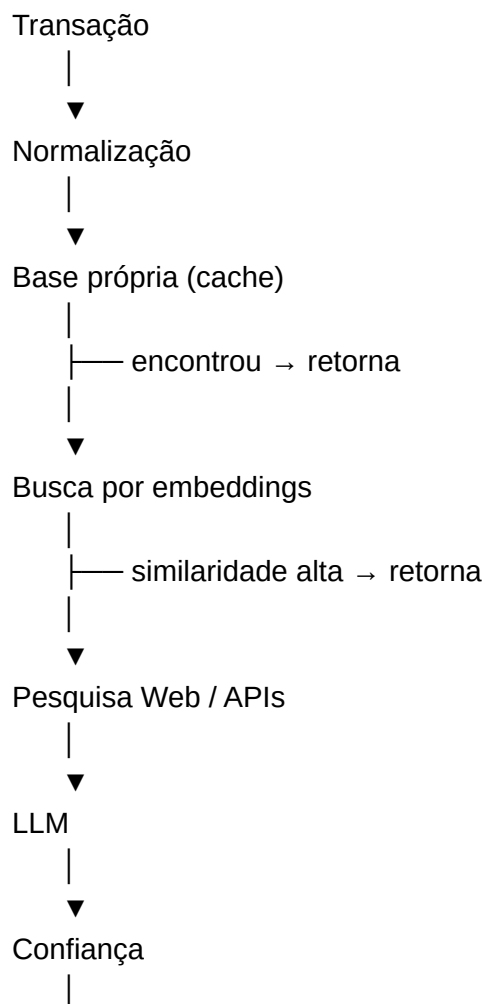
Categoria:
Desconhecida

Confiança:
12%

Ação:
Solicitar classificação do usuário.

Essa abordagem evita classificações erradas.

Na minha experiência, uma arquitetura próxima desta costuma dar os melhores resultados:



- └─ alta → salva na base
- └─ baixa → revisão humana

Essa estratégia reduz drasticamente o custo com LLMs e aumenta a precisão, porque o modelo deixa de "adivinhar" categorias para estabelecimentos desconhecidos e passa a decidir com base em evidências. É exatamente o tipo de arquitetura adotada por muitas plataformas financeiras e de gestão de despesas.

e qual ferramenta e boa pra isso ?

RouteLLM

Para esse problema específico, eu não escolheria uma única ferramenta. O ideal é combinar algumas tecnologias. Se eu fosse construir hoje um classificador de faturas para produção, consideraria algo assim:

Função	Ferramenta recomendada
Orquestração do agente	LangGraph
LLM	GPT-5.5, Claude 4 ou Gemini 2.5 Pro
Embeddings	OpenAI (text-embedding-3-large) ou BGE
Banco vetorial	pgvector (PostgreSQL) ou Qdrant
Busca na web	Tavily ou SerpAPI
Extração de páginas	Firecrawl
Banco de conhecimento	PostgreSQL
Observabilidade	LangSmith

Eu dividiria em três categorias:

1. Se você quer algo robusto para produção

Eu iria de:

- LangGraph
- PostgreSQL + pgvector
- OpenAI ou Claude

- LangSmith

O LangGraph é excelente porque permite criar um fluxo como:

Recebe estabelecimento

↓

Normaliza

↓

Existe no banco?

↓

Sim → retorna

↓

Não

↓

Busca similar

↓

Achou?

↓

Sim → retorna

↓

Não

↓

Pesquisa Web

↓

LLM decide

↓

Salva no banco

Esse tipo de fluxo é exatamente o que o LangGraph faz muito bem.

1. Se você quer algo mais simples

CrewAI.

Ele permite criar agentes como:

- Pesquisador
- Classificador
- Validador

É muito simples de desenvolver.

O problema é que quando o fluxo cresce (cache, retries, human-in-the-loop, etc.), o LangGraph costuma escalar melhor.

1. Se você quer máxima precisão

Eu faria um sistema de RAG (Retrieval-Augmented Generation).

Por exemplo:

Recebe

"POSTO SAO JOSE LTDA"

↓

Embedding

↓

Busca 20 estabelecimentos parecidos

↓

LLM recebe:

- posto shell
- posto ipiranga
- posto são josé

↓

Categoria:
Combustível

Confiança:
98%

Nesse cenário, o LLM não precisa "conhecer" o estabelecimento. Ele só precisa comparar com exemplos semelhantes.

Esse padrão costuma funcionar muito bem.

Uma ferramenta que pouca gente usa e que pode ser um diferencial é o Google Maps.

Imagine receber:

BAR DO ZE LTDA

O agente consulta o Google Maps e encontra:

Bar do Zé



Categoria:
Bar

Serve almoço

Pronto.

O LLM praticamente só confirma.

Isso costuma ser muito mais confiável do que pedir para o modelo adivinhar a categoria apenas pelo nome.

Se fosse montar esse projeto hoje do zero, minha stack seria:

- Backend: Python + FastAPI
- Agentes: LangGraph
- Banco: PostgreSQL
- Vetores: pgvector
- LLM: GPT-5.5 ou Claude 4
- Busca Web: Tavily
- Observabilidade: LangSmith
- Cache: Redis

Essa arquitetura é escalável, relativamente simples de manter e permite incorporar novas fontes de informação ao longo do tempo.

Uma última observação: para classificação de estabelecimentos, muitas equipes descobrem que o maior ganho não vem de usar um LLM mais avançado, mas de construir uma boa base de conhecimento com cache, busca semântica e enriquecimento de dados. Depois de alguns meses de operação, a maior parte das transações passa a ser classificada pela própria base, deixando o LLM apenas para os casos realmente novos.

queria tambem baixar bastante os custos usando modelos open source

RouteLLM

Nesse caso, eu mudaria um pouco a arquitetura. Se o objetivo é reduzir custo ao máximo, você pode chegar muito perto da qualidade de modelos comerciais usando open source, desde que use recuperação de contexto e um bom pipeline.

Uma stack que eu recomendaria seria:

- Orquestração: LangGraph
- LLM: Qwen3, DeepSeek ou Llama 3.3
- Embeddings: BGE-M3 ou bge-large-en-v1.5 (ou um modelo multilíngue se houver muitos nomes em português)
- Banco vetorial: Qdrant ou PostgreSQL + pgvector
- Reranker: BGE Reranker
- Banco relacional: PostgreSQL

O maior erro que vejo é fazer:

Nome:

SUPERMERCADO SILVA LTDA

↓

LLM

↓

Categoria

Nenhum modelo consegue manter conhecimento sobre milhões de estabelecimentos.

O ideal é transformar o problema em busca + decisão.

Exemplo:

SUPERMERCADO SILVA LTDA

↓

embedding

↓

Qdrant

↓

20 estabelecimentos parecidos

↓

LLM

O prompt fica parecido com:

Estabelecimento:

SUPERMERCADO SILVA LTDA

Mais parecidos:

Supermercado Silva

Categoria: Mercado

Supermercado Silva Express

Categoria: Mercado

Mercado Silva

Categoria: Mercado

Escolha a categoria mais provável.

Até um modelo de 7B consegue acertar muito bem nesse cenário.

Alguns modelos open source que eu considero fortes hoje:

Modelo	Qualidade	Velocidade	RAM (4-bit)
Qwen3 8B	★★★★★	★★★★★	~6 GB
Qwen3 14B	★★★★★	★★★★☆	~10 GB
DeepSeek-R1 Distill 8B	★★★★☆	★★★★☆	~6 GB
Llama 3.3 70B	★★★★★	★★☆☆☆	>40 GB
Gemma 3 12B	★★★★☆	★★★★☆	~9 GB

Hoje, o Qwen3 8B costuma oferecer uma excelente relação entre custo, velocidade e qualidade para tarefas desse tipo.

Outra ideia interessante é usar modelos menores para as tarefas simples e só recorrer a modelos maiores quando necessário. Por exemplo:

Nova transação

↓

Existe no banco?

↓

Sim

↓

Fim

Não

↓

Busca vetorial

↓

Similaridade > 0.97?

↓

Sim

↓

Classifica automaticamente

↓

Não

↓

Qwen3 8B

↓

Confiança > 90%?

↓

Sim

↓

Salva

↓

Não

↓

Qwen3 32B ou Claude/GPT

Assim, talvez 95% das transações sejam resolvidas pelo banco de conhecimento, 4% por um modelo open source de 8B e apenas 1% por um modelo maior.

Também vale considerar uma etapa de normalização antes de tudo. Um modelo pequeno pode transformar:

AMZN MKTPLACE*1234

em

Amazon

ou

IFOOD*BURGERKING

em

Burger King

Esse tipo de pré-processamento aumenta bastante a eficácia da busca vetorial.

Se você pretende hospedar tudo localmente, uma combinação bastante equilibrada seria:

- Inferência: vLLM ou Ollama (para desenvolvimento) e vLLM para produção.
- Embeddings: BGE-M3.
- Banco vetorial: Qdrant.
- Orquestração: LangGraph.

Uma pergunta importante: qual o volume de transações que você precisa classificar? Estamos falando de algo como 10 mil por dia, 1 milhão por dia ou dezenas de milhões por mês? A resposta muda bastante a arquitetura recomendada.

e no caso de modelos mais leves pra rodar em maquina propria no caso eu tenho uma rtx 3060

RouteLLM

Com uma RTX 3060 (12 GB de VRAM), você consegue rodar modelos muito bons para classificação de faturas. A chave é escolher modelos de 7B a 14B quantizados (GGUF ou AWQ/GPTQ), que cabem confortavelmente na GPU.

Minha recomendação seria:

Modelo	Tamanho	RTX 3060	Recomendação
Qwen3 8B	8B	★★★★★★	Melhor escolha
Gemma 3 4B	4B	★★★★★★	Muito rápido
Gemma 3 12B	12B	★★★★★	Boa qualidade
Llama 3.1/3.2 8B	8B	★★★★★	Muito estável
DeepSeek-R1 Distill 8B	8B	★★★★★	Bom para raciocínio

Para classificação, eu evitaria modelos gigantes como 32B ou 70B, porque o ganho de qualidade geralmente não compensa o aumento de custo computacional.

O que eu usaria:

- LLM: Qwen3 8B Instruct
- Embeddings: BGE-M3
- Vetores: Qdrant
- Framework: LangGraph

Isso roda tranquilamente em uma 3060.

Uma arquitetura eficiente seria:

Transação

↓

Normalização

↓

Cache PostgreSQL

↓

Embedding (BGE-M3)

↓

Qdrant

↓

Top 10 semelhantes

↓

Qwen3 8B

↓

Categoria

↓

Salva

Nesse fluxo, o modelo quase nunca precisa "inventar" uma categoria, apenas escolher entre informações recuperadas.

Quanto à inferência:

- Para desenvolvimento: Ollama é extremamente simples.
- Para produção: vLLM (se o modelo for suportado) ou SGLang oferecem melhor desempenho.

Você também pode usar LM Studio para testes rápidos.

Se optar pelo Ollama, basta baixar um modelo e começar:

```
ollama run qwen3:8b
```

ou

```
ollama run gemma3:4b
```

Para classificação, eu daria preferência ao Qwen3 8B.

Outra técnica que pode fazer bastante diferença é o fine-tuning. Se você tiver algumas dezenas de milhares de exemplos classificados, pode fazer um LoRA sobre um modelo como Qwen3 8B. Isso costuma ser bem mais barato do que consultar

um modelo comercial e faz o modelo aprender padrões específicos da sua base (como nomes abreviados de estabelecimentos ou categorias próprias).

Minha recomendação para sua máquina seria:

- GPU: RTX 3060 12 GB
- Modelo: Qwen3 8B Instruct (quantizado em 4 bits)
- Embeddings: BGE-M3
- Banco vetorial: Qdrant
- Banco relacional: PostgreSQL
- Orquestração: LangGraph
- Fine-tuning: LoRA, se você tiver um conjunto de dados rotulado.

Essa configuração oferece um excelente equilíbrio entre desempenho, custo e facilidade de operação em hardware local.

queria tambem um ambiente mais rapido ainda sim tenho pouca ram 32gb e boa parte esta ocupada ja por outros processos ate mesmo esses modelos deixam a maquina lenta ja que e muita coisa rodando em docker

RouteLLM

Nesse cenário, eu pensaria diferente. O gargalo não é o modelo, é a infraestrutura.

Você tem:

- RTX 3060 12 GB (boa GPU)
- 32 GB de RAM
- Muitos containers Docker
- Pouca RAM livre
- Quer baixa latência

Eu evitaria rodar um ecossistema com PostgreSQL + Qdrant + LangGraph + Redis + Ollama + APIs, porque isso pode consumir muitos gigabytes de RAM e deixar a máquina lenta.

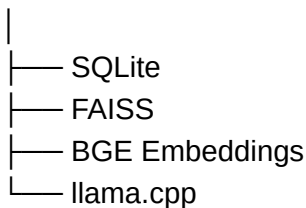
Uma abordagem mais enxuta seria:

- Um único container para a aplicação (FastAPI)
- Um único processo de inferência (Ollama ou llama.cpp)
- SQLite para cache (ou PostgreSQL se você já usa)
- FAISS em vez de Qdrant

O FAISS é uma biblioteca, não um servidor. Você carrega o índice diretamente na memória do processo, eliminando um container inteiro.

Ficaria algo como:

FastAPI



São apenas dois processos principais: sua aplicação e o modelo.

Outra ideia é evitar um framework de agentes quando ele não agrega valor. Para classificação de faturas, muitas vezes um pipeline simples é suficiente:

1. Normalizar texto
2. Consultar cache
3. Buscar similares
4. Chamar LLM (se necessário)
5. Salvar resultado

Esse fluxo pode ser implementado em poucas centenas de linhas de Python, sem LangGraph ou CrewAI. Frameworks são excelentes para fluxos complexos, mas para um pipeline fixo eles adicionam dependências e consumo de memória.

Sobre o modelo, eu também escolheria algo menor:

- Gemma 3 4B: muito rápido e leve.
- Qwen3 4B (se disponível na variante desejada): boa relação entre desempenho e consumo.
- Qwen3 8B: se ainda couber confortavelmente na sua VRAM.

Como a inferência fica na GPU, o principal consumo de RAM vem do restante da aplicação e dos serviços auxiliares.

Uma otimização importante é usar modelos quantizados em 4 bits (Q4_K_M ou equivalente). Eles reduzem bastante o uso de VRAM e RAM com perda pequena de qualidade para esse tipo de tarefa.

Se quiser ir ainda mais longe na economia de recursos, você pode separar as tarefas:

- Um modelo pequeno (4B) faz normalização e classificação inicial.
- Apenas casos de baixa confiança vão para um modelo maior (8B).

Na prática, se 90% das transações forem resolvidas pelo modelo menor, a máquina fica muito mais responsiva.

Se eu tivesse exatamente o seu hardware e precisasse manter o ambiente leve, minha stack seria:

- FastAPI
- SQLite (ou PostgreSQL se já existir)
- FAISS
- sentence-transformers com um modelo de embeddings leve
- llama.cpp com um modelo quantizado de 4B ou 8B
- Sem Redis
- Sem Qdrant
- Sem LangGraph
- Sem múltiplos containers para componentes que podem rodar na mesma aplicação

Isso reduz significativamente o consumo de memória e a complexidade operacional.

Uma curiosidade: quantos containers Docker você costuma manter ativos ao mesmo tempo? E quanto de RAM fica realmente livre antes de iniciar o modelo? Essas duas informações ajudam a definir se vale a pena um modelo de 8B ou se um de 4B será a escolha mais equilibrada.

e como eu posso criar bases de conhecimento com o tempo

RouteLLM

Na verdade, esse é o ponto onde você pode construir um diferencial competitivo. O LLM é apenas uma peça; a base de conhecimento é o que melhora continuamente o sistema.

Eu trataria cada classificação como uma oportunidade de aprendizado.

Imagine esta tabela:

estabelecimentos

id
nome_original
nome_normalizado
categoria
subcategoria
confianca
fonte
ultima_atualizacao
qtd_transacoes

Exemplo:

IFOOD*BURGERKINGRJ

↓

burger king

↓

Alimentação

↓

Fast Food

↓

0.99

↓

Google Maps

Na próxima vez que aparecer:

IFOOD*BKCOPA

Você normaliza para:

burger king

e já retorna a classificação.

Isso é a primeira camada da base de conhecimento.

Depois evolua para aliases

Burger King

aliases:

BK

BURGERKING

BURGER KING

IFOOD BURGER KING

BURGER KING RJ

BK COPACABANA

Todos apontam para o mesmo estabelecimento.

Depois armazene embeddings

Além do nome:

Burger King

embedding:

[0.1234, 0.9876...]

Quando chega

BURGUER KING

ou

BK SHOPPING

o embedding encontra a mesma empresa.

Depois armazene contexto

Não guarde apenas a categoria.

Guarde informações úteis.

```
{
  "nome": "Burger King",
  "categoria": "Fast Food",
  "site": "https://burgerking.com.br",
  "cidade": "São Paulo",
  "pais": "Brasil",
  "tipo": "Restaurante",
  "fonte": "Google Maps",
  "tags": [
    "hamburger",
    "lanche",
    "alimentação"
  ]
}
```

Depois o LLM recebe isso.

Aprenda com o usuário

Essa é provavelmente a parte mais importante.

Se o sistema disser:

Amazon

↓

Shopping

e o usuário alterar para

Eletrônicos

você salva:

Amazon

↓

Eletrônicos

↓

Usuário confirmou

Agora aquela classificação vale muito mais.

Faça enriquecimento automático

Um agente roda toda madrugada.

Ele pega os estabelecimentos desconhecidos.

XYZ LTDA

↓

Google

↓

Receita Federal

↓

Google Maps

↓

Site

↓

Categoria

↓

Salva

Sem intervenção humana.

Crie uma memória por estabelecimento

Não apenas:

Posto Shell

↓

Combustível

Mas também:

Total de vezes visto:

15.438

Última compra:

02/07/2026

Categoria confirmada:

98%

Cidade:

Curitiba

Ticket médio:

R\$ 212

Isso aumenta muito a confiança.

Use RAG

A base pode ser pesquisada pelo agente.

Usuário



Mercado Novo Horizonte



Busca vetorial



20 estabelecimentos semelhantes



LLM



Categoria

Quanto maior sua base, melhor o sistema fica.

Tenha uma fila de aprendizado

Classificação



Confiança >95%



Vai direto para a base

Confiança entre 70 e 95%

↓

Vai para revisão

Confiança <70%

↓

Pesquisa Web

↓

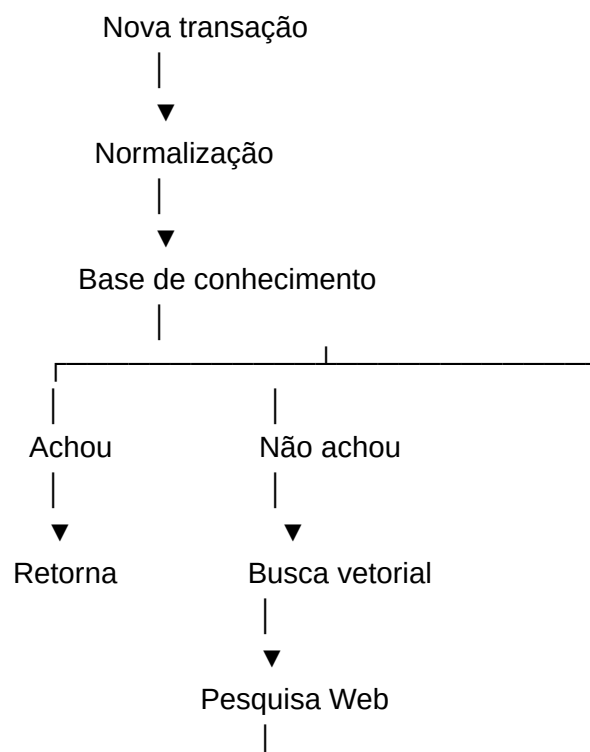
LLM

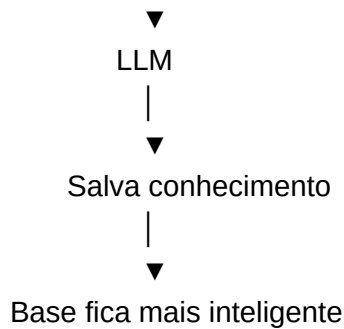
↓

Revisão humana

Assim a base cresce com qualidade.

Minha arquitetura seria algo como:





O interessante é pensar na base como um ativo. Depois de alguns meses, ela pode conter centenas de milhares de estabelecimentos já classificados. Nessa fase, a maioria das transações será resolvida consultando a base, e apenas uma pequena fração precisará de um LLM.

Se você classifica transações de empresas brasileiras, há ainda uma oportunidade adicional: enriquecer a base com dados públicos, como informações de CNPJ (quando disponíveis), CNAE, município e nome fantasia. Essas informações ajudam bastante a distinguir estabelecimentos com nomes semelhantes e podem melhorar a precisão da classificação.

sentence transformer usa base de dados ?

RouteLLM

A resposta curta é: **Não diretamente**. O Sentence Transformer (ST) é apenas o "tradutor", mas ele geralmente trabalha junto com uma base de dados vetorial para ser útil.

Para entender fácil, pense assim:

- **O Sentence Transformer:** É um modelo (como o paraphrase-multilingual-MiniLM-L12-v2) que pega um texto ("PADARIA DO ZE") e transforma em uma lista de números chamada **Embedding** (ex: [0.12, -0.05, 0.88...]).
- **A Base de Dados:** É onde você guarda esses números para pesquisar depois.

Como eles trabalham juntos na prática:

Passo 1: Criando a Base (Indexação)

1. Você pega 10.000 faturas já classificadas.

2. Passa cada nome pelo **Sentence Transformer**.
3. Salva o número (vetor) e a categoria em uma base (pode ser o **FAISS**, que é leve, ou **Qdrant**).

Passo 2: Consultando (Busca Semântica)

1. Chega uma fatura nova: "PAG*ZE PADARIA".
2. Você passa esse nome pelo **mesmo Sentence Transformer**.
3. Ele gera um vetor.
4. Você pergunta para a base de dados: *"Qual dos 10.000 vetores que eu já tenho é o mais parecido com este?"*
5. A base retorna: "PADARIA DO ZE" (Confiança: 98%).

Por que usar Sentence Transformers no seu caso?

No seu problema de faturas, os nomes mudam mas o sentido é o mesmo.

- Uma busca comum (SQL LIKE) falharia ao comparar "IFOOD * BK" com "BURGER KING".
- O **Sentence Transformer** entende que eles "moram perto" no espaço matemático e consegue dizer que são a mesma coisa.

Ferramentas recomendadas para o seu ambiente leve:

Como você quer economizar RAM e tem uma RTX 3060:

1. **Modelo:** Use o sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2 ou o sentence-transformers/clip-ViT-B-32-multilingual-v1. Eles são minúsculos (menos de 500MB) e rodam muito rápido na CPU ou GPU.
2. **Base de Dados:**
 - **FAISS:** É uma biblioteca do Facebook. Não é um "banco de dados" que fica rodando um serviço (Docker). Você só importa no Python (import faiss). Ele guarda os vetores na memória RAM ou em um arquivo .index. É a opção mais rápida para 32GB de RAM ocupados.

- **SQLite + ChromaDB:** Se quiser algo que pareça mais um banco de dados, mas que rode "dentro" do seu código sem precisar de Docker separado.

Exemplo de código (Conceito):

```
from sentence_transformers import SentenceTransformer
import faiss

# 1. Carrega o modelo (O "Tradutor")
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# 2. Seus dados conhecidos
nomes_conhecidos = ["PADARIA DO ZE", "POSTO IPIRANGA", "MERCADO LIVRE"]
categorias = ["Alimentação", "Transporte", "Compras"]

# 3. Transforma em números e guarda no FAISS (A "Base")
embeddings = model.encode(nomes_conhecidos)
index = faiss.IndexFlatL2(embeddings.shape[1])
index.add(embeddings)

# 4. Chega uma fatura nova
nova_fatura = "PAG*ZE PADARIA LTDA"
novo_embedding = model.encode([nova_fatura])

# 5. Busca o mais próximo
D, I = index.search(novo_embedding, 1)

# Retorna o índice do mais parecido
print(f"Parece ser: {nomes_conhecidos[I[0][0]]} - Categoria: {categorias[I[0][0]]}")
```

Resumo para você: Use o Sentence Transformer para gerar a "digital" do estabelecimento e o **FAISS** para guardar essas digitais em um arquivo simples no seu disco, sem precisar de mais um container Docker pesado.

queria um exemplo com Langgraph pgvector e SentenceTransformer

RouteLLM

This request is better suited for Abacus AI Agent. Switching you over.

Click below to continue in Abacus AI Agent.

Redirecting to Abacus AI Agent. Convo link is shown in the UI for the user to click and redirect to Abacus AI Agent.

quero so um exemplo de codigo python

RouteLLM

Segue um exemplo completo integrando **LangGraph** (orquestração), **pgvector** (busca vetorial no PostgreSQL) e **SentenceTransformer** (embeddings) para classificação de faturas.

Estrutura do fluxo

Transação → Normaliza → Busca no pgvector →
|— Achou (similaridade alta) → retorna categoria
|— Não achou → LLM classifica → salva no pgvector

1. Setup do banco (rode uma vez)

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
CREATE TABLE estabelecimentos (  
  id SERIAL PRIMARY KEY,  
  nome_normalizado TEXT NOT NULL,  
  categoria TEXT NOT NULL,  
  confianca REAL DEFAULT 1.0,  
  embedding vector(384) -- 384 = dim do MiniLM  
);
```

```
CREATE INDEX ON estabelecimentos  
  USING hnsw (embedding vector_cosine_ops);
```

2. Código Python

```

import re
import psycopg2
from typing import TypedDict, Optional
from sentence_transformers import SentenceTransformer
from langgraph.graph import StateGraph, END

# ----- Configuração -----
DB_CONFIG = {
    "host": "localhost",
    "dbname": "faturas",
    "user": "postgres",
    "password": "postgres",
}
SIMILARITY_THRESHOLD = 0.85

# cosine similarity mínima para reaproveitar

model = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")

# ----- Estado do grafo -----
class State(TypedDict):
    nome_original: str
    nome_normalizado: str
    embedding: list
    categoria: Optional[str]
    confianca: float
    fonte: str

# ----- Utilitários de banco -----
def get_conn():
    return psycopg2.connect(**DB_CONFIG)

# ----- Nós do grafo -----
def normalizar(state: State) -> State:
    txt = state["nome_original"].lower()
    txt = re.sub(r"(ifood|pag|pix|amzn|mktplace)*?", "", txt)
    txt = re.sub(r"\b(ltda|me|epp|sa|s/a)\b", "", txt)
    txt = re.sub(r"^[a-z0-9\s]", " ", txt)
    txt = re.sub(r"\s+", " ", txt).strip()
    state["nome_normalizado"] = txt
    state["embedding"] = model.encode(txt).tolist()
    return state

```

```

def buscar_vetorial(state: State) -> State:
    emb = state["embedding"]
    with get_conn() as conn, conn.cursor() as cur:
        # 1 - (distância cosseno) = similaridade cosseno
        cur.execute(
            """
            SELECT categoria, 1 - (embedding <=> %s::vector) AS sim
            FROM estabelecimentos
            ORDER BY embedding <=> %s::vector
            LIMIT 1
            """,
            (emb, emb),
        )
        row = cur.fetchone()

    if row and row[1] >= SIMILARITY_THRESHOLD:
        state["categoria"] = row[0]
        state["confianca"] = float(row[1])
        state["fonte"] = "pgvector"
    else:
        state["categoria"] = None
    return state

```

```

def classificar_llm(state: State) -> State:
    # Substitua por chamada real (Ollama, vLLM, OpenAI, etc.)
    categoria = chamar_llm(state["nome_normalizado"])
    state["categoria"] = categoria
    state["confianca"] = 0.9
    state["fonte"] = "llm"
    return state

```

```

def salvar(state: State) -> State:
    with get_conn() as conn, conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO estabelecimentos
            (nome_normalizado, categoria, confianca, embedding)
            VALUES (%s, %s, %s, %s::vector)
            """,
            (
                state["nome_normalizado"],
                state["categoria"],
                state["confianca"],
            )
        )

```

```

        state["embedding"],
    ),
)
conn.commit()
return state

```

```

def chamar_llm(nome: str) -> str:
    """Stub — troque por Ollama/vLLM/OpenAI."""
    import ollama
    prompt = f"""Classifique o estabelecimento em UMA categoria
(Alimentação, Transporte, Compras, Saúde, Lazer, Outros).
Estabelecimento: "{nome}"
Responda apenas com a categoria."""
    resp = ollama.chat(
        model="qwen3:8b",
        messages=[{"role": "user", "content": prompt}],
    )
    return resp["message"]["content"].strip()

```

```

# ----- Roteamento condicional -----
def rota_apos_busca(state: State) -> str:
    return "fim" if state["categoria"] else "llm"

```

```

# ----- Montagem do grafo -----
def build_graph():
    g = StateGraph(State)
    g.add_node("normalizar", normalizar)
    g.add_node("buscar", buscar_vetorial)
    g.add_node("llm", classificar_llm)
    g.add_node("salvar", salvar)

    g.set_entry_point("normalizar")
    g.add_edge("normalizar", "buscar")
    g.add_conditional_edges(
        "buscar", rota_apos_busca, {"fim": END, "llm": "llm"}
    )
    g.add_edge("llm", "salvar")
    g.add_edge("salvar", END)
    return g.compile()

```

```

# ----- Uso -----
if __name__ == "__main__":

```

```
app = build_graph()
```

```
for fatura in ["IFOOD*BURGER KING RJ", "POSTO SHELL LTDA", "PAG*ZE PADARIA"]:  
    resultado = app.invoke({"nome_original": fatura})  
    print(  
        f"{fatura:30} -> {resultado['categoria']:15} "  
        f"(conf: {resultado['confianca']:.2f}, fonte: {resultado['fonte']})"  
    )
```

Pontos-chave

- `<=>` é o operador de distância cosseno do pgvector; 1 - distância dá a similaridade.
- O índice **HNSW** deixa a busca rápida mesmo com milhões de registros.
- Toda transação nova classificada pelo LLM é **salva automaticamente**, então a base cresce sozinha e o LLM é chamado cada vez menos.
- O `SIMILARITY_THRESHOLD` controla o quão "confiante" a busca precisa ser antes de reaproveitar uma categoria — ajuste conforme seus testes.

Se quiser, posso adaptar o nó chamar_llm para **vLLM** ou adicionar uma camada de **revisão humana** (human-in-the-loop) para os casos de baixa confiança. Quer?

usando from langchain_postgres.vectorstores import PGVector

RouteLLM

Segue o exemplo adaptado usando PGVector do langchain_postgres:

1. Setup do banco (rode uma vez)

```
CREATE EXTENSION IF NOT EXISTS vector;
```

O PGVector cria a tabela automaticamente.

2. Dependências

```
pip install langchain-postgres sentence-transformers langgraph psycopg2-binary
```

3. Código Python

```
import re
from typing import TypedDict, Optional
from sentence_transformers import SentenceTransformer
from langchain_postgres.vectorstores import PGVector
from langchain_core.documents import Document
from langchain_core.embeddings import Embeddings
from langgraph.graph import StateGraph, END

# ----- Configuração -----
CONNECTION_STRING =
"postgresql+psycopg2://postgres:postgres@localhost:5432/faturas"
COLLECTION_NAME = "estabelecimentos"
SIMILARITY_THRESHOLD = 0.85

# ----- Wrapper do SentenceTransformer para LangChain -----
class STEmbeddings(Embeddings):
    def __init__(self, model_name: str = "paraphrase-multilingual-MiniLM-L12-v2"):
        self.model = SentenceTransformer(model_name)

    def embed_documents(self, texts: list[str]) -> list[list[float]]:
        return self.model.encode(texts).tolist()

    def embed_query(self, text: str) -> list[float]:
        return self.model.encode(text).tolist()

embeddings = STEmbeddings()

vectorstore = PGVector(
    embeddings=embeddings,
    collection_name=COLLECTION_NAME,
    connection=CONNECTION_STRING,
    use_jsonb=True,
)

# ----- Estado do grafo -----
class State(TypedDict):
    nome_original: str
```

```
nome_normalizado: str
categoria: Optional[str]
confianca: float
fonte: str
```

```
# ----- Nós do grafo -----
```

```
def normalizar(state: State) -> State:
    txt = state["nome_original"].lower()
    txt = re.sub(r"(ifood|pag|pix|amzn|mktplace)\*?", "", txt)
    txt = re.sub(r"\b(ltda|me|epp|sa|s/a)\b", "", txt)
    txt = re.sub(r"^[a-z0-9\s]", " ", txt)
    txt = re.sub(r"\s+", " ", txt).strip()
    state["nome_normalizado"] = txt
    return state
```

```
def buscar_vetorial(state: State) -> State:
    resultados = vectorstore.similarity_search_with_relevance_scores(
        query=state["nome_normalizado"],
        k=1,
    )
```

```
    if resultados:
        doc, score = resultados[0]
        if score >= SIMILARITY_THRESHOLD:
            state["categoria"] = doc.metadata["categoria"]
            state["confianca"] = float(score)
            state["fonte"] = "pgvector"
            return state
```

```
    state["categoria"] = None
    state["confianca"] = 0.0
    return state
```

```
def classificar_llm(state: State) -> State:
    categoria = chamar_llm(state["nome_normalizado"])
    state["categoria"] = categoria
    state["confianca"] = 0.9
    state["fonte"] = "llm"
    return state
```

```
def salvar(state: State) -> State:
    doc = Document(
```

```

        page_content=state["nome_normalizado"],
        metadata={
            "nome_original": state["nome_original"],
            "categoria": state["categoria"],
            "confianca": state["confianca"],
        },
    )
    vectorstore.add_documents([doc])
    return state

```

```

def chamar_llm(nome: str) -> str:
    """Stub — troque por Ollama/vLLM/OpenAI."""
    import ollama
    prompt = f"""Classifique o estabelecimento em UMA categoria
    (Alimentação, Transporte, Compras, Saúde, Lazer, Outros).
    Estabelecimento: "{nome}"
    Responda apenas com a categoria."""
    resp = ollama.chat(
        model="qwen3:8b",
        messages=[{"role": "user", "content": prompt}],
    )
    return resp["message"]["content"].strip()

```

```

# ----- Roteamento condicional -----
def rota_apos_busca(state: State) -> str:
    return "fim" if state["categoria"] else "llm"

```

```

# ----- Montagem do grafo -----
def build_graph():
    g = StateGraph(State)
    g.add_node("normalizar", normalizar)
    g.add_node("buscar", buscar_vetorial)
    g.add_node("llm", classificar_llm)
    g.add_node("salvar", salvar)

    g.set_entry_point("normalizar")
    g.add_edge("normalizar", "buscar")
    g.add_conditional_edges(
        "buscar", rota_apos_busca, {"fim": END, "llm": "llm"}
    )
    g.add_edge("llm", "salvar")
    g.add_edge("salvar", END)
    return g.compile()

```

```

# ----- Popular base inicial (opcional) -----
def popular_base():
    exemplos = [
        ("burger king", "Alimentação"),
        ("mcdonalds", "Alimentação"),
        ("posto shell", "Transporte"),
        ("posto ipiranga", "Transporte"),
        ("netflix", "Lazer"),
        ("spotify", "Lazer"),
        ("amazon", "Compras"),
        ("mercado livre", "Compras"),
        ("farmacia drogasil", "Saúde"),
    ]
    docs = [
        Document(
            page_content=nome,
            metadata={"categoria": cat, "confianca": 1.0},
        )
        for nome, cat in exemplos
    ]
    vectorstore.add_documents(docs)
    print(f"{len(docs)} estabelecimentos adicionados à base.")

# ----- Uso -----
if __name__ == "__main__":
    # popular_base() # descomente na primeira execução

    app = build_graph()

    faturas = [
        "IFOOD*BURGER KING RJ",
        "POSTO SHELL LTDA",
        "PAG*ZE PADARIA",
        "NETFLIX.COM",
        "AMZN MKTPLACE*1234",
    ]

    for fatura in faturas:
        resultado = app.invoke({"nome_original": fatura})
        print(
            f'{fatura:35} -> {resultado["categoria"]:15} '
            f'(conf: {resultado["confianca"]:.2f}, fonte: {resultado["fonte"]})'
        )

```

Principais diferenças em relação à versão anterior

Ponto	Versão anterior	Esta versão
Busca vetorial	psycopg2 + SQL manual	PGVector do LangChain
Inserção	SQL manual	vectorstore.add_documents()
Embeddings	Direto no nó	Wrapper STEmbeddings
Tabela	Criada manualmente	Criada automaticamente

Saída esperada

IFOOD*BURGER KING RJ -> Alimentação (conf: 0.97, fonte: pgvector)
POSTO SHELL LTDA -> Transporte (conf: 0.94, fonte: pgvector)
PAG*ZE PADARIA -> Alimentação (conf: 0.72, fonte: llm)
NETFLIX.COM -> Lazer (conf: 0.99, fonte: pgvector)
AMZN MKTPLACE*1234 -> Compras (conf: 0.96, fonte: pgvector)

Quer que eu adicione também a camada de **revisão humana** para casos de baixa confiança ou um endpoint **FastAPI** para expor isso como uma API?
